dataframe_revision

February 22, 2026

Read

```
df = spark.read.csv("path/file.csv", header=True, inferSchema=True,sep=",")
```

Best Practice

```
df = spark.read \
          .option("header", True) \
          .option("inferSchema", True) \
          .csv("path/file.csv")
```

BASIC COMMANDS

Task	Command
Show data	`df.show()`
Print schema	`df.printSchema()`
Columns list	`df.columns`
Count rows	`df.count()`
Select columns	`df.select("col1","col2")`
Filter rows	`df.filter(condition)`
OR filter	`df.filter((cond1) | (cond2))`
AND filter	`df.filter((cond1) & (cond2))`
Distinct	`df.select("col").distinct()`
Drop column	`df.drop("col")`
Rename column	`df.withColumnRenamed("old","new")`
Sort ascending	`df.orderBy("col")`
Sort descending	`df.orderBy(col("col").desc())`
Limit N rows	`df.limit(5)`

COLUMN FUNCTIONS

```
from pyspark.sql.functions import *
```

Function	Use
col(“name”)	Reference column
lit(100)	Add constant value
alias(“new”)	Rename output column
when(cond, val)	Conditional logic
otherwise(val)	Else condition

Function	Use
concat(col1,col2)	Combine columns
upper() / lower()	Case conversion
length()	String length
round(col,2)	Round values

example:

when+otherwise

```
df.withColumn("status",
              when(col("amount") > 100000, "High")
              .otherwise("Low"))
```

AGGREGATION PATTERNS

Question Type	Pattern
Count per key	`df.groupBy("key").count()`
Sum per key	`df.groupBy("key").sum("col")`
Multiple agg	`df.groupBy("key").agg(sum("col"), avg("col"))`
Rename agg column	`agg(sum("col").alias("total"))`
Average	`avg("col")`
Max	`max("col")`
Min	`min("col")`

Multi Aggregation Template

```
df.groupBy("state").agg(
    count("*").alias("total_orders"),
    sum("amount").alias("total_revenue"),
    avg("amount").alias("avg_revenue")
)
```

Having condition(dataframe doesnt have direct having) so use filter after aggregation

```
df.groupBy("custId") \
  .sum("amount") \
  .filter(col("sum(amount)") > 100000)
```

JOIN PATTERNS

Join Type	Syntax
Inner Join	`df1.join(df2, "id", "inner")`
Left Join	`"left"`
Right Join	`"right"`
Full Join	`"outer"`
Anti Join	`"left_anti"`

Ex:

```
df1.join(df2, df1.id == df2.id, "inner")
```

DATE FUNCTIONS(IMP)

Function	Use
current_date()	Today's date
current_timestamp()	Current timestamp
to_date(col)	Convert to date
year(col)	Extract year
month(col)	Extract month
dayofmonth(col)	Extract day
datediff(date1,date2)	Difference in days
date_add(col,10)	Add days
date_sub(col,5)	Subtract days

Ex:

```
df.withColumn("year", year(col("order_date")))
```

CREATE/MODIFY COLUMN PATTERNS

Task	Pattern
Add new column	`withColumn("new", expression)`
Conditional column	`when().otherwise()`
Multiply columns	`col("qty") * col("price")`
Cast type	`col("age").cast("int")`

NULL HANDLING

Task	Command / Pattern
Check NULL	`col("col").isNull()`
Check NOT NULL	`col("col").isNotNull()`
Filter NULL rows	`df.filter(col("col").isNull())`
Filter NOT NULL rows	`df.filter(col("col").isNotNull())`
Drop rows with ANY null	`df.na.drop()`
Drop rows (specific column)	`df.na.drop(subset=["col"])`
Drop rows where ALL columns null	`df.na.drop(how="all")`
Fill all nulls with 0	`df.na.fill(0)`
Fill specific column	`df.na.fill({"col": 0})`
Fill multiple columns	`df.na.fill({"col1":0,"col2":"Unknown"})`
Replace specific value	`df.na.replace("NA", None)`
Replace null using when	`when(col("col").isNull(), val).otherwise(col("col"))`
First non-null value (SQL coalesce)	`coalesce(col("c1"), col("c2"))`
Count NULL values	`count(when(col("col").isNull(), True))`

WINDOW FUNCTIONS

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

windowSpec = Window.partitionBy("state").orderBy(col("amount").desc())

df.withColumn("rank", row_number().over(windowSpec))
```

EXAM PATTERNS

```
# Total revenue per product
df.withColumn("revenue", col("qty")*col("price")) \
  .groupBy("product") \
  .sum("revenue")

# Customers with > 2 orders
df.groupBy("custId") \
  .count() \
  .filter(col("count") > 2)

# Top 3 products by revenue
df.groupBy("product") \
  .sum("revenue") \
  .orderBy(col("sum(revenue)").desc()) \
  .limit(3)

# Anti Join (customers with no orders)
customers.join(orders, "custId", "left_anti")
```

Function	Meaning
`df.coalesce(1)`	Reduce partitions → single output file
`coalesce(col1,col2)`	Return first non-null column value

Coalesce Function (VERY IMPORTANT)

Different from coalesce(1) for partitions.

This coalesce() is SQL function.

Returns first non-null value.

```
df.select(coalesce(col("phone"), lit("Not Available")))
```

always use from pyspark.sql.functions import *

Question Contains	Immediately Use
“Total”	sum()
“Count”	count()
“Average”	avg()
“More than”	filter()

Question Contains	Immediately Use
“Top N”	orderBy(desc) + limit()
“Never”	left_anti join
“Year wise”	year()
“Add column”	withColumn()
“Condition column”	when()

Question Phrase	What To Use
“Remove null rows”	`df.na.drop()`
“Replace null with 0”	`df.na.fill(0)`
“Replace null in one column”	`df.na.fill({"col": value})`
“Check missing values”	`isNull()`
“Use backup column if null”	`coalesce(col1,col2)`

UDF(User defined functions)

`Used when built-in functions are not enough.`

```
# Step 1: Define Python Function
def age_category(age):
    if age >= 60:
        return "Senior"
    else:
        return "Adult"

# Step 2: Register UDF
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

age_udf = udf(age_category, StringType())

# Step 3: Apply UDF
df = df.withColumn("category", age_udf(col("age")))
```

Defining own schema

```
# Define Schema
from pyspark.sql.types import *

schema = StructType([
    StructField("custId", StringType(), True),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("state", StringType(), True)
])
```

```
# Read with Schema
df = spark.read \
          .option("header", True) \
          .schema(schema) \
          .csv("customers.csv")
```

Write Data

```
# Basic Write
df.write.csv("output_path")

# Write with Header
df.write \
  .option("header", True) \
  .csv("output_path")

# Overwrite Mode
df.write \
  .mode("overwrite") \
  .csv("output_path")

Modes:
"overwrite"
"append"
"ignore"
"error"
```